Série TP 4

Exercice1 : class Point

1) Ecrire une classe *Point* qui permet de représenter un point sur un plan cartésien. Chaque point est caractérisé par son abscisse et son ordonnée (de type float), une méthode *afficher()* imprimant sur l'écran ses coordonnées et une méthode *déplacer()* effectuant son déplacement selon les pas communiqués à la méthode comme arguments (déplacement par rapport à la position courante). **Remarque :** on utilisera le constructeur par défaut.

2) Ecrire une classe *TestPoint* décrivant un petit programme utilisant la classe *Point*. Ce programme permet de créer un point, en afficher ses coordonnées, le déplacer (abscisse +2.5 et ordonnée -1) et en afficher de nouveau ses coordonnées.
3) Reprendre la définition de la classe *Point* en ajoutant :

- un constructeur sans paramètres qui initialise les coordonnées à (0,0)
- un constructeur recevant en arguments les coordonnées d'un point
- une méthode *toString()* permettant de retourner une chaîne de caractères (un String) décrivant un point (dans ce cas ses coordonnées)
- une méthode *compareTo()* qui retourne un résultat booléen selon que les attributs de l'objet point communiqué en argument possèdent les même valeurs que ceux de l'objet courant ou non.

4) Modifier la classe *TestPoint* de façon à créer deux objets *Point*, le premier avec le constructeur paramétré (abscisse =1 et ordonnée =2.3) et le deuxième avec le constructeur sans paramètres. Le premier point doit être déplacé (abscisse –0.5 et ordonné +0.7), ensuite, le deuxième point doit être confondu (même position) avec le premier point. Afficher les coordonnées des deux points (de deux façons).
5) Comparer les deux points crées précédemment en appliquant l'opérateur == sur leurs références puis en faisant l'appel à la méthode *compareTo()* de *Point*. Refaites la comparaison avec la méthode *compareTo()* après la rendre *static*.

Exercice2 : class Cercle

Un cercle est caractérisé par son centre (*Point**) et son rayon (*float*). Chaque cercle peut : afficher la position de son centre, retourner la valeur de son rayon, se déplacer, changer de taille (rayon doit rester strictement positif), calculer et retourner la valeur de sa surface, calculer et retourner la valeur de son périmètre, et finalement retourner une description de ses caractéristiques (y compris la surface et le périmètre).

1) Définir la classe *Cercle* en fournissant deux versions pour le constructeur : *public Cercle (float, Point)* et *public Cercle(float, float, float)*.
2) Définir une classe *TestCercle* qui permet de :
   - Créer un cercle *c* selon les caractéristiques fournies par l'utilisateur, afficher ses caractéristiques, le déplacer vers l'origine, mettre son rayon à *1* et enfin afficher ses caractéristiques de nouveau.
   - Créer un point *p* dans un endroit demandé par l'utilisateur et créer deux cercles concentriques *c1* et *c2* de rayons différents et ayant *p* comme centre. Le programme doit afficher la position du cercle *c1*, déplacer *c2* avec des pas indiqués par l'utilisateur puis afficher de nouveau la position du cercle *c1*.

Exercice3 : class Segment

Un segment de droite est caractérisé par deux points qui représentent ses extrémités (extrémité gauche et extrémité droite). Un segment de droite peut avoir plusieurs comportements dont on cite dans cet exercice :

- Se déplacer par rapport à l'axe des abscisses (sans changement de longueur)
- Se déplacer par rapport à l'axe des ordonnées (sans changement de longueur)
- Calculer et retourner la valeur de sa longueur selon la formule suivante :

$$sqrt((extrDroit.abscisse - extrGauche.abscisse)^2 + (extrDroit.ordonne - extrGauche.ordonne)^2)$$

- Retourner une description sur ses caractéristiques, y compris sa longueur

1) Définir la classe *Segment* qui définit un segment de droite dont le constructeur reçoit en argument les deux points des extrémités
2) Définir la classe *TestSegment* qui :
   - Crée deux points différents *p1* et *p2*, un segment *s* ayant *p1* et *p2* comme extrémités et deux cercles *c1* et *c2* (définition de *Cercle* de l'exercice1) dont les centres sont respectivement *p1* et *p2*.
   - Affiche les caractéristiques de *s*, *c1* et *c2*
   - Déplace *s* selon l'axe des abscisses puis selon l'axe des ordonnées par des pas communiqués par l'utilisateur, avec un affichage des caractéristiques de *c1* et *c2* à l'issu de chaque déplacement
   - Déplace *c2* par des pas communiqués par l'utilisateur puis affiche la longueur du segment *s*